

From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration

Sthitadhi Sengupta
George Mason University
Fairfax, VA, USA
ssengup2@gmu.edu

Krish Jana*
Cornell University
Ithaca, NY, USA
kj364@cornell.edu

Nan Wu
George Mason University
Fairfax, VA, USA
nwu5@gmu.edu

Songqing Chen
George Mason University
Fairfax, VA, USA
sqchen@gmu.edu

Matteo Varvello
Nokia Bell Labs
Murray Hill, NJ, USA
matteo.varvello@nokia.com

Bo Han
George Mason University
Fairfax, VA, USA
bohan@gmu.edu

Abstract

With the rising demand for cost-effective and privacy-preserving deep learning and visualization services, service providers are increasingly turning to in-browser solutions. General-purpose computations, leveraging the graphics processing unit (GPU), are foundational to executing algorithms that power these services. Web graphics library (WebGL) is a widely adopted GPU-access application programming interface (API) designed for multidimensional rendering in browsers, while WebGPU is a newer API developed with compute-specific capabilities. Although WebGPU is a promising standard, its performance has not been systematically evaluated for general-purpose computation. This paper investigates WebGPU and WebGL for accelerating client-side computation in web browsers. By benchmarking key computational GPU kernels for 16 PolyBench and 2 CHStone functions, we measure the performance of WebGPU and WebGL across varying input sizes and algorithmic complexities. Our results show that: 1) both WebGPU and WebGL exhibit poorer performance than central processing unit (CPU)-based execution for small input data due to setup and CPU-GPU synchronization overheads, but they outperform CPU execution as the input data size increases; 2) WebGL performs better than WebGPU for small inputs, except for CPU-driven loop functions, due to its lower initial setup overhead; and 3) WebGPU outperforms WebGL for large inputs through optimized GPU thread utilization and achieves better performance for loop-driven algorithms across all input sizes by minimizing CPU-GPU data exchange. Overall, our results indicate that WebGPU is a competitive option for enhancing the execution performance of large-scale web applications.

CCS Concepts

• **Networks** → **Network measurement**; • **Information systems** → **Web applications**;

Keywords

Measurement, GPU acceleration, Web performance, WebGPU, WebGL, WebAssembly

ACM Reference Format:

Sthitadhi Sengupta, Nan Wu, Matteo Varvello, Krish Jana*, Songqing Chen, and Bo Han. 2025. From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration. In *Proceedings of the 2025 ACM Internet Measurement Conference (IMC '25)*, October 28–31, 2025, Madison, WI, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3730567.3764504>

1 Introduction

Web applications enable service providers to deliver complex functionality to a broad and diverse set of end-users directly through browsers, ensuring seamless execution on various devices. These applications are becoming increasingly sophisticated, incorporating myriad techniques for image processing and deep learning [26, 29, 42]. General-purpose computations are a fundamental prerequisite for popular image processing and deep learning applications, and they benefit substantially from swift execution in web browsers [42]. Addressing computational bottlenecks is, therefore, crucial to enhancing web application performance and better serving end-users.

Graphics processing unit (GPU)-based acceleration has enhanced the performance of a diverse range of applications in non-web browser settings, such as machine learning, database systems, image processing, and scientific simulations [12, 15, 23, 32]. Effective utilization of the GPU within the client-side web browser can play a key role in alleviating computational bottlenecks while simultaneously reducing server-side costs [26]. In addition, executing applications on the client-side strengthens privacy by eliminating the need to transmit personal data to external servers. Thus, GPU acceleration of general-purpose computations, combined with their client-side execution within the web browser environment, represents an important advancement.

Web graphics library (WebGL) and WebGPU application programming interfaces (APIs) can harness the GPU to accelerate the execution of general-purpose computations in browsers [40]. WebGL is primarily known for enabling multidimensional rendering in browsers [2], whereas WebGPU is a newer API designed to better support general-purpose computations. In WebGL, computations are carried out through *graphics-oriented shaders*, while in WebGPU they are implemented using *compute-oriented shaders* [2, 6–8]. Both WebGL and WebGPU can be programmed in C or C++ and executed on the web by compiling the code to JavaScript or WebAssembly using, for example, the Emscripten compiler [1, 2, 6, 42].

*Work was done when at Montville Township High School.



This work is licensed under a Creative Commons Attribution 4.0 International License. *IMC '25, Madison, WI, USA*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1860-1/2025/10
<https://doi.org/10.1145/3730567.3764504>

Although recent work has examined the performance of general-purpose computations using WebAssembly in a web browser environment, GPU acceleration for these functions has not yet been investigated [42]. In this paper, we present a measurement study of JavaScript and WebAssembly alongside their WebGPU- and WebGL-integrated counterparts for general-purpose computations using PolyBench and CHStone, two benchmark suites of functions written in C [4, 25]. These suites provide a wide variety of computations, enabling us to generalize key performance insights relevant to software developers. Our study reveals the following findings regarding the performance of WebGL and WebGPU:

- While computations using GPU-access APIs execute >95% faster or exhibit a consistently slower rate of execution time growth than those running entirely on the CPU for large input data sizes, CPU-based computations execute >100% faster for small inputs. The behavior observed for large inputs is expected, as the GPU's parallel architecture executes independent operations faster than the CPU. However, for small input data, the setup and CPU-GPU synchronization overheads of both APIs dominate, making CPU execution preferable over employing GPU acceleration.
- WebGL runs >25% faster than WebGPU for small input data because WebGPU's setup time is double that of WebGL; the exception to this trend occurs when a central CPU loop orchestrates the function. While WebGL only incurs a context setup overhead, WebGPU experiences both adapter and device setup overheads.
- For large input data, the performance of GPU-access APIs is determined primarily by the GPU execution time. WebGPU provides fine-grained control over GPU thread workgroup optimization, leading to faster GPU execution compared to WebGL. This is demonstrated by WebGPU outperforming WebGL by >10% in certain cases. In other cases, WebGPU sustains a low execution time growth with increasing input data sizes despite its higher initial overheads. Moreover, for loop-orchestrated functions, WebGPU runs >37% faster than WebGL, also observed for small input sizes. WebGL incurs higher overhead for these functions due to repeatedly transferring data between the CPU and GPU, as it lacks the ability to store intermediate results on the GPU across loop iterations. WebGPU's better thread- and data-management model makes it the stronger choice for both large-scale hybrid workloads and computations that require CPU-GPU synchronizations.
- The performance gains of WebAssembly over JavaScript are less pronounced for WebGL and WebGPU than for CPU-based execution, where WebAssembly achieves >40% speedup. The two languages exhibit comparable execution-time performance across both GPU-access APIs, with outcomes ranging from neutral to either language having a slight advantage. Overall, WebAssembly outperforms JavaScript in a greater number of cases by <40% for GPU-access APIs. Since the primary bottlenecks in WebGPU and WebGL lie on the GPU side, they are largely unimpacted by WebAssembly's faster CPU-side execution. Consequently, developer efforts should primarily focus on GPU-side optimization.

We believe these findings will provide valuable guidance for software developers in optimizing web applications efficiently. Fundamentally, at least a subset of the algorithms developed for an application should be parallelizable (*i.e.*, capable of executing independent operations simultaneously) if either GPU-access APIs are

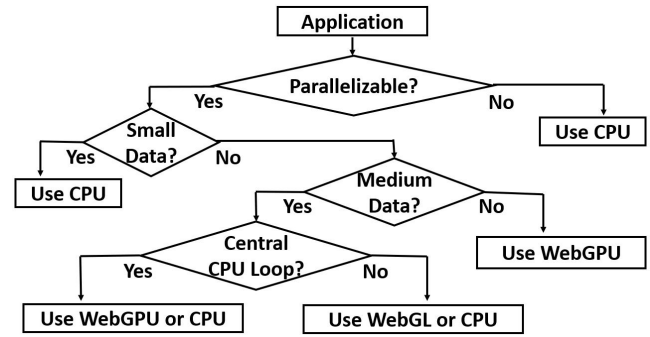


Figure 1: Implementation Choice Decision-Tree for Software Developers.

to be employed. Moreover, these APIs are not well-suited for applications that process small data. In this scenario, CPU represents the best execution choice.

There are gray areas for input data sizes, where they are neither too small nor too large, and determining which API, if any, to use becomes challenging. Fortunately, most real-world applications handle massive datasets, which makes WebGPU the rule-of-thumb choice. Additionally, if an application has a central CPU loop coordinating execution and involves extensive interdependent data across iterations, WebGL should be dismissed as an option, irrespective of input size. However, developers must also evaluate whether the input data size is sufficiently small for WebGPU's initialization overhead and CPU-GPU synchronization costs to have a negative impact on computation performance. In such scenarios, the CPU may be the preferred choice.

Furthermore, for these gray-area input sizes, functions without centralized loop orchestration but containing parallelizable portions present a challenging scenario in which developers must weigh the bottlenecks inherent in WebGL and WebGPU against the more predictable execution of the CPU, while also considering the potential benefits of parallel execution for independent computations.

Figure 1 succinctly illustrates and guides the key choices developers will need to make. Developers are strongly recommended to minimize thread workgroup utilization and maximize the threads invoked per workgroup when parallelizing algorithms in WebGPU to achieve optimal performance.

Also, our results imply that the developers' choice of web programming language (JavaScript or WebAssembly) has minimal impact when the majority of the workload is on the GPU side.

2 Background

Web Applications refer to programs that are stored on a remote server and then distributed over a network to end users using web browsers [38]. For example, deep learning neural networks can be deployed and accessed by end users on client devices through web browsers [26, 29]. JavaScript is a popular programming language for building web applications but cannot fully utilize the capabilities of both the CPU and GPU [26]. A measurement study has shown that WebAssembly may be a better language choice for building web applications due to its compact and low-level characteristics [42].

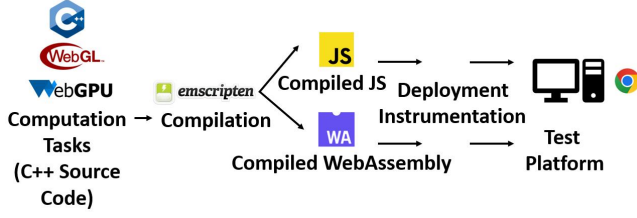


Figure 2: Overview of Our Measurement Process.

General Purpose Computation encompasses applications that rely on vector and matrix operations to perform a wide range of tasks [17]. The computational time may be a bottleneck when loading a web application in a mobile browser [33]. Synchronization, the coordination of concurrent processes to enable system operation, is an essential paradigm in GPU-accelerated general-purpose computing and can incur large overheads [28].

WebGL is a graphics API that can be used to perform multidimensional rendering functions in a web browser. It is based on OpenGL for Embedded Systems (ES) 2.0 [2]. WebGL2 is the most updated version of WebGL and allows more advanced features and functionalities. WebGL requires the setup of the context and shader program as important steps to run in a browser environment. The context encapsulates the state of the current WebGL instance [3, 5]. Although WebGL is compatible across most web browsers, it has not kept pace with hardware advances due to its reliance on the OpenGL ES design [14]. This can be an issue, as end users may access the web application from diverse platforms.

WebGPU is an API that provides standardized access to GPUs and is oriented for web usage. Compared to existing lower-level GPU-access APIs such as Vulkan and DirectX, WebGPU grants increased abstraction and broader usability across devices and platforms [6]. However, WebGPU is under active development, with full compatibility currently limited to Google Chrome and Microsoft Edge. WebGPU includes a shader specifically designed for compute tasks [6, 7], a feature not supported in WebGL. WebGPU requires setting up the adapter, device, and shader module [6]. The device is the primary object for interactions with WebGPU and forms the foundation of object creation, GPU communications, and error management [6]. An adapter acts as an interface for a device’s GPU; a device with more than one GPU may offer a corresponding number of adapters [6]. The command queue is responsible for organizing and submitting commands to the GPU for execution purposes [6].

3 Methodology

Overview. Figure 2 summarizes the procedure to measure the performance of JavaScript and WebAssembly with WebGL and WebGPU. There are 4 steps: (1) *Code Conversion*, (2) *Integrating C++ Code with GPU-Access APIs*, (3) *Compiling to JavaScript or WebAssembly*, and (4) *Data Collection*.

We identify multiple functions, listed in Table 1, the vast majority of which are drawn from the PolyBench suite and written in C, that are most likely to benefit from the parallelization capabilities of GPU-access APIs [4, 25]. We first rewrite each chosen PolyBench function in C++, implementing the CPU, WebGL and WebGPU versions. C++ provides a cleaner interface and structure for programming with WebGPU while also maintaining execution

Function	Description
2mm	Two matrix multiplications
3mm	Three matrix multiplications
bicg	Biconjugate gradient stabilized method
covariance	Normalized covariance
mvt	Matrix vector multiplication
syr2k	Symmetric rank 2k update
gesummv	Summed matrix-vector multiplications
atax	Matrix transpose and vector multiplication
doitgen	Multiresolution analysis kernel
durbin	Toeplitz system solver
gramschmidt	Gram-schmidt decomposition
deriche	Deriche edge detection and smoothing
fdtd-2d	2D finite difference time domain kernel
heat-3d	Heat equation over 3D space
jacobi-1d	1D Jacobi stencil computation
jacobi-2d	2D Jacobi stencil computation
sha-256	Secure hash algorithm
aes-128	Advanced encryption standard

Table 1: List of Measured Functions.

speed comparable to C. Emscripten [1], an LLVM/ Clang-based compiler, is then used to compile the CPU, WebGL, and WebGPU versions of each C++ function into JavaScript and WebAssembly. Finally, we record the execution time of each function by averaging across five runs.

We also evaluate two complex cryptographic functions, using manually-written JavaScript (*i.e.*, JavaScript code written directly rather than compiled from C/C++) for both WebGL and WebGPU, to diversify our areas of assessment. The two functions, SHA-256 [21] and AES-128 [11], are drawn from the CHStone benchmark [25] and are chosen primarily for their widespread use in cybersecurity. Collectively, these 18 functions represent a heterogeneous set of computations that are of paramount importance to the functioning of web-based applications. We use a Windows 11 Home Alienware computer with an AMD Ryzen 9 5900 12-core processor and NVIDIA RTX 3080 GPU for our measurements. The experiments were conducted using the Google Chrome browser (version 135.0.7049.42) in April 2025.

3.1 Code Conversion

We use functions from the PolyBench and CHStone suites [4, 25], which were originally written in C. We transform the PolyBench source code into C++, replacing PolyBench arrays with native array constructs. We use float instead of double for input data types and instrument the C++ code with high-resolution execution timers.

3.2 Integrating C++ Code with GPU-Access APIs

After transforming the functions from C to C++, we integrate them with the GPU-access APIs. We implement the parallelizable parts of the functions in shader code, which executes on the GPU. For WebGL, we use the GLSL shader language [3], while for WebGPU, we employ WGSL [8]. Generally, the parallelizable parts of a code are loops performing independent operations on a matrix or vector. For WebGPU, we write WGSL code to be used by a compute shader. However, for WebGL, we write GLSL code to be utilized

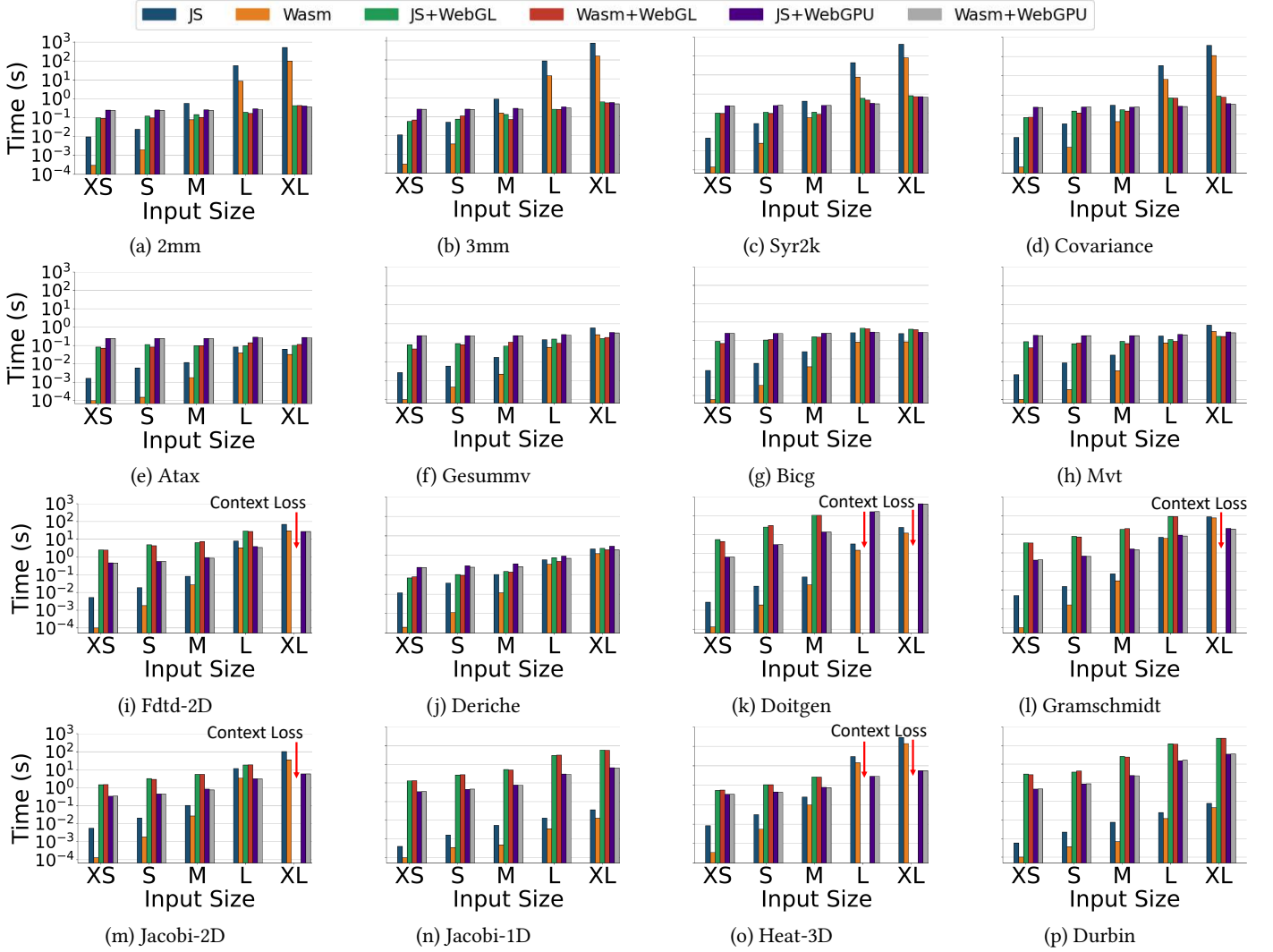


Figure 3: Execution Time for PolyBench Functions with Different Input Data Sizes for Six Configurations.

by a fragment shader since it cannot leverage compute shaders. For WebGPU, we seek to minimize the WGSL parameter *thread workgroups*, which represents the set of threads launched for parallel operations, while simultaneously maximizing the number of threads that can be invoked per workgroup, which are determined during the setup phase. The C++ portions of the code for both WebGL and WebGPU execute on the CPU, initializing input matrices, vectors, and variables for GPU transfer and setting up objects that encapsulate the synchronization and data exchange processes with the GPU.

3.3 Compiling to JavaScript or WebAssembly

We use the Emscripten compiler to compile each function in our source code into six different configurations: *CPU-based JavaScript*, *CPU-based WebAssembly*, *WebGL integrated with JavaScript*, *WebGL integrated with WebAssembly*, *WebGPU integrated with JavaScript*,

and *WebGPU integrated with WebAssembly*. Emscripten automatically converts the C++ syntax to JavaScript and WebAssembly syntax. For WebGL, we write the code in OpenGL (C++), which Emscripten compiles into both JavaScript- and WebAssembly-integrated versions of WebGL. We use five different input sizes for the six configurations of each function: *Extra Small (XS)*, *Small (S)*, *Medium (M)*, *Large (L)*, and *Extra Large (XL)*. These input sizes are adopted from prior work that evaluates the web performance of these functions on the CPU using WebAssembly [42]. Emscripten ensures that all source code is compiled along with the corresponding HTML files required to run them in a web browser.

3.4 Data Collection

For data collection, we enable WebGL and WebGPU-related flags for Google Chrome due to browser restrictions that exist for these GPU-access APIs. We measure the *execution time* of each function in the six configurations mentioned above. Execution time refers to the time it takes to complete the computational task in the

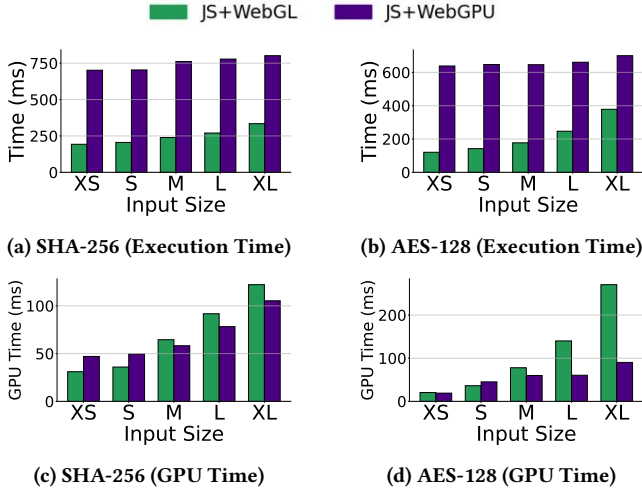


Figure 4: Performance of CHStone Cryptographic Functions.

browser. We achieve high-resolution function runtimes by embedding fine-grained timers directly in the precompiled C++ source code, thus minimizing measurement overhead. We ensure that only one browser tab is open while running each benchmark. We run each benchmark five times and calculate the average of these runs.

4 Measurement Results

For computational tasks, execution time plays a critical role in shaping the user experience. Lower execution time indicates better performance, as it improves user experience by reducing lag. Therefore, we conduct an analysis that involves the execution time in the Google Chrome browser.

The bar plots in Figure 3 are created by scaling the execution time values. We divide the raw execution time by 0.0001 and then compute the base-10 logarithm of the result, using this scale to better illustrate data trends. Also, the WebGL implementations of certain functions, marked “Context Loss” in Figure 3, exhaust the dedicated GPU memory for large input data, resulting in context loss during browser execution. Given that this limitation arises from a commonly available “in the wild” CPU-GPU combination, we refrain from upgrading to a more powerful GPU with larger memory solely to obtain results for the WebGL implementations of these functions. As most users rely on commodity GPUs rather than specialized high-end ones, it is important that our experiments focus on this representative class of devices.

4.1 GPU-Access APIs vs CPU

We notice that for all functions, as input size increases, both WebGPU and WebGL typically execute >95% faster or display a slower rate of execution time growth than CPU-only execution of computations, because of the GPU’s ability to parallelize independent operations. However, for small input data, they perform worse than CPU implementations of algorithms, which execute the computations >100% faster. Although initially counterintuitive, further investigation reveals a constant delay of approximately 0.2 seconds in the early stages of WebGPU’s execution; for WebGL, this delay is 0.1 seconds. We discover that this delay is caused by the adapter and device setups for WebGPU, while for WebGL, it is the

context setup. For WebGPU, both setup times are roughly 0.1 seconds each. Meanwhile, for WebGL, the context setup is 0.1 seconds. These bottlenecks are particularly evident in functions such as 2mm, Syr2k, and 3mm, where nearly all computations are offloaded to the GPU. As the actual GPU execution time for these functions is relatively small with small input data, the setup overhead becomes more pronounced. These performance bottlenecks remain the same regardless of the input data size.

Functions such as Jacobi-2D, Heat-3D, Fdtd-2D, Jacobi-1D, Gramschmidt, Durbin, and Doitgen exhibit slower execution in their GPU-access API versions compared to CPU implementations for small input data sizes, as they contain a central loop on the CPU where computations are dispatched to the GPU at each iteration. This design necessitates synchronization between the CPU and GPU, and the resulting synchronization overhead degrades performance for small data sizes as the overhead accumulates over multiple loop iterations.

4.2 WebGPU vs WebGL

We observe that WebGL typically executes >25% faster than WebGPU for small input data, with loop-driven functions being the exception. This behavior arises because, at these input sizes, both WebGL and WebGPU are constrained by their setup time. However, WebGPU’s combined setup time, influenced by both adapter and device initializations, is approximately twice that of WebGL’s context setup.

For large input sizes, however, the performance is dominated by GPU execution time, as setup overhead becomes relatively negligible. WebGPU enables explicit control over the number of threads invoked per workgroup, allowing us to better exploit parallelism within its functions. In contrast, WebGL does not provide such flexibility. WebGPU outperforms WebGL by >10% for functions such as Covariance, Bigg, and Doitgen for large inputs. In cases where WebGPU underperforms or achieves comparable performance to WebGL for large input sizes, the limitation stems from its inability to overcome the initial setup overhead within the tested input size range. Nevertheless, WebGPU exhibits a slower rate of execution time growth with input data size compared to WebGL, as observed in functions such as 2mm, 3mm, Syr2k, Atax, and Mvt.

Manually-written JavaScript exhibits the same behavior as compiled JavaScript for both WebGPU and WebGL. When evaluating advanced cryptographic algorithms, as shown in Figure 4, we find that WebGPU incurs a higher overall execution time than WebGL. However, WebGPU’s GPU execution time is lower for larger input data sizes, and WebGL’s GPU execution time increases rapidly with input data size. Although WebGPU performs worse overall for the tested input size ranges, its slower execution time growth and lower GPU execution time for larger input data suggest that it will outperform WebGL at even greater data sizes for these pertinent cryptographic algorithms.

Finally, we observe that for centralized loop-based functions such as Jacobi-1D, Jacobi-2D, Fdtd-2D, Durbin, Doitgen, and Heat-3D, WebGPU runs >37% faster than WebGL. WebGL’s poor performance for this class of functions arises not only because of CPU-GPU synchronization, but also from the additional overhead of transferring the entire data between the CPU and GPU at every iteration of the central loop, an intrinsic limitation of WebGL.

4.3 JavaScript vs WebAssembly

WebAssembly executes <40% faster than JavaScript across both GPU-access APIs. However, there are several cases where the performance is comparable between the two, or where JavaScript outperforms WebAssembly by a margin of <35%. While WebAssembly yields marginal improvements, particularly for large input sizes, we do not observe the major gains seen in CPU-only execution, where WebAssembly executes >40% faster than JavaScript. This difference arises because the primary bottlenecks for both APIs lie on the GPU side. The setup processes for WebGPU, GPU execution time, and synchronization are unaffected by the rapid CPU-side code execution provided by WebAssembly. The marginal improvements observed in functions likely stem from the acceleration of CPU-side tasks, such as matrix initializations.

5 Discussion

Threats to Validity. Our study is threatened on the following grounds: the device on which the algorithms are measured, and future updates for WebGPU and WebGL. The device architecture and the relative capabilities of its CPU and GPU may lead to varying results. For example, when context loss occurs for loop-centric functions due to the inefficiency of WebGL, the input size at which it happens may differ depending on the device. In addition to that, WebGPU and WebGL are APIs that are constantly being optimized and modified. The results may differ in future versions of the GPU-access APIs, particularly if upcoming iterations of WebGL incorporate compute shaders similar to those in WebGPU. We note that our findings reflect only the versions of WebGPU and WebGL available as of April 2025.

Future Work. Our current work primarily focuses on algorithms from the PolyBench suite. While we briefly evaluate the performance of two important cryptographic algorithms (SHA-256 and AES-128), future research could yield new insights by more thoroughly examining WebGPU-accelerated versions of complex cryptographic and simulation applications. Furthermore, if compute shaders are incorporated into a future version of WebGL, its performance should also be systematically benchmarked.

6 Related Work

Performance Evaluation and Optimization for Browser Applications. Our work is closely related to those that explore performance evaluation, optimization, and general-purpose computation for browser applications [13, 27, 31, 35, 42]. For example, Yan *et al.* [42] benchmark equivalent WebAssembly and JavaScript applications on multiple web browsers to examine performance differences. Sapuan *et al.* [35] present `gpu.js`, an abstracted and portable Javascript framework for general-purpose GPU computing in browsers that provides an alternative to CUDA and OpenCL. Baek *et al.* [13] develop Wasm-R3, a framework that records and optimizes execution traces of WebAssembly modules and then replays them, and ultimately introduce the Wasm-R3-Bench benchmark. Jangda *et al.* [27] build Browsix-Wasm, an extension enabling UNIX APIs to run in web environments, and discover performance degradation in applications compiled to WebAssembly when testing the SPEC CPU suite across multiple browsers. To the best of our knowledge, our work conducts the first performance measurement of

general-purpose computation utilizing WebGPU and WebGL in a web browser environment.

Graphics Performance with GPU-Access APIs. Previous work has investigated the performance of GPU-access APIs for graphics applications [14, 16, 19, 20, 24, 34, 36, 37, 39]. For example, Han *et al.* [24] enhance web performance for graphics rendering applications such as MotionMark1.3 by designing an interceding middleware to dynamically reconfigure WebGL code into WebGPU during execution. Bi *et al.* [14] utilize WebGPU and draw call optimizations to improve graphics rendering in web browsers. Chickerur *et al.* [16] benchmark WebGPU and WebGL's performance for graphical operations such as cube rotation and particles scattering in a Web 3.0 environment. Palmér [34] evaluates the performance of WebGL and WebGPU for 2D particle systems across two GPUs of varying compute power and observes that WebGPU achieves superior graphics computation, supporting a greater number of particles than WebGL. Fransson *et al.* [20] compare the rasterization performance of WebGL and WebGPU as backend languages for the Godot game engine [9] and find that WebGPU reduces the per-frame processing time across the CPU and GPU. While prior studies have primarily focused on the performance of graphics applications using GPU-access APIs, our work is the first to measure these APIs for general-purpose computations.

Web Performance for Deep Learning. Earlier research efforts have also focused on optimizing or measuring the performance of deep learning applications in a web browser [18, 22, 26, 29, 30, 41]. For instance, Ma *et al.* [29] evaluate the performance of web browser-based deep learning applications using seven JavaScript frameworks. Wang *et al.* [41] report a deep learning inference performance study in a browser environment, also examining its quality of experience (QoE). In contrast, we evaluate general-purpose computation performance in the web browser as a foundation for applications beyond deep learning.

7 Conclusion

In this paper, we conduct the first measurement study of the performance of WebGPU and WebGL for general-purpose computations in a web browser environment. We measure the performance across 18 PolyBench and CHStone functions when utilizing WebGPU, WebGL, WebAssembly, and JavaScript on Google Chrome. We discover that small input data sizes do not require GPU-access APIs for efficient execution, whereas large inputs benefit significantly from WebGPU. Moreover, for GPU acceleration, the choice of web programming language becomes less critical, although WebAssembly generally provides marginally better overall performance than JavaScript. We hope our work encourages further exploration into algorithmic optimization for web-based computing environments.

Acknowledgments

We thank the anonymous reviewers and the shepherd for their constructive and actionable feedback. We also extend our sincere appreciation to Jonathan Ding, Jing Bao, Fanchen Kong, and Jie Chen from Intel for their invaluable assistance in addressing technical issues of WebGPU and WebGL. This work was partially supported by the National Science Foundation under Grant CNS-2235049.

References

- [1] 2024. Emscripten Documentation. https://emscripten.org/docs/getting_started/index.html. [accessed on 09/30/2025].
- [2] 2024. OpenGL: The Industry's Foundation for High Performance Graphics. <https://www.khronos.org/opengl/>. [accessed on 09/30/2025].
- [3] 2024. OpenGL Wiki. <https://wikis.khronos.org/opengl/>. [accessed on 09/30/2025].
- [4] 2024. PolyBenchC-4.2.1. <https://github.com/MatthiasJReisinger/PolyBenchC-4.2.1>. [accessed on 09/30/2025].
- [5] 2024. WebGL2 Rendering Context. <https://developer.mozilla.org/en-US/docs/Web/API/WebGL2RenderingContext>. [accessed on 09/30/2025].
- [6] 2024. WebGPU C++ guide. <https://eliemichel.github.io/LearnWebGPU/introduction.html>. [accessed on 09/30/2025].
- [7] 2024. WebGPU Explainer. <https://gpuweb.github.io/gpuweb/explainer/>. [accessed on 09/30/2025].
- [8] 2024. WebGPU Shading Language. <https://www.w3.org/TR/WGSL/>. [accessed on 09/30/2025].
- [9] 2025. Godot Game Engine. <https://godotengine.org/>. [accessed on 09/30/2025].
- [10] 2025. WebGPU Compute Shader Basics. <https://webgpufundamentals.org/webgpu/lessons/webgpu-compute-shaders.html>. [accessed on 09/30/2025].
- [11] Ria Andriani, Ema Wijayanti, and Wahyu Wibowo. 2018. Comparison of AES 128, 192 and 256 Bit Algorithm for Encryption and Description File. In *Proceedings of International Conference on Information Technology, Information Systems and Electrical Engineering*. doi:10.1109/ICITISEE.2018.8720983
- [12] Andreas Athanasopoulos, Anastasios Dimou, Vasileios Mezaris, and Ioannis Kompatsiaris. 2011. GPU Acceleration for Support Vector Machines. In *Proceedings of International Workshop on Image Analysis for Multimedia Interactive Services*. <https://resolver.tudelft.nl/uuid:6716875f-5b40-4e7b-9f9d-24a85c02ee3b> [accessed on 09/30/2025].
- [13] Doehyun Baek, Jakob Getz, Yusung Sim, Daniel Lehmann, Ben L. Titzer, Sukyoung Ryu, and Michael Pradel. 2024. Wasm-R3: Record-Reduce-Replay for Realistic and Standalone WebAssembly Benchmarks. *Proceedings of the ACM on Programming Languages* 8 (2024), 2156–2182. doi:10.1145/3689787
- [14] Weichen Bi, Yun Ma, Yudong Han, Yifan Chen, Deyu Tian, and Jiaqi Du. 2024. FusionRender: Harnessing WebGPU's Power for Enhanced Graphics Performance on Web Browsers. In *Proceedings of ACM on Web Conference*. doi:10.1145/3589334.3645395
- [15] Sebastian Breß, Max Heimel, Norbert Siegmund, Ladjel Bellatreche, and Gunter Saake. 2014. GPU-Accelerated Database Systems: Survey and Open Challenges. *Transactions on Large-Scale Data- and Knowledge-Centered Systems XV* 8920 (2014), 1–35. doi:10.1007/978-3-662-45761-0_1
- [16] Satyadhyam Chickerur, Sankalp Balannavar, Pranali Hongekar, Aditi Prerna, and Soumya Jituri. 2024. WebGL vs WebGPU: A Performance Analysis for Web 3.0. In *Proceedings of Innovative Data Communication Technologies and Application*. doi:10.1016/j.procs.2024.03.281
- [17] Wu chun Feng and Shuai Xiao. 2010. To GPU Synchronize or Not GPU Synchronize. In *Proceedings of IEEE International Symposium on Circuits and Systems*. doi:10.1109/ISCAS.2010.5537722
- [18] Zainul Abi Din, Panagiotis Tigas, Samuel T. King, and Benjamin Livshits. 2020. PERCIVAL: Making In-Browser Perceptual Ad Blocking Practical with Deep Learning. In *Proceedings of USENIX Annual Technical Conference*. doi:10.48550/arXiv.1905.07444
- [19] Landon Dyken, Pravin Poudel, Will Usher, Steve Petruzza, Jake Y. Chen, and Sidharth Kumar. 2022. GraphWaGu: GPU Powered Large Scale Graph Layout Computation and Rendering for the Web. In *Proceedings of Eurographics Symposium on Parallel Graphics and Visualization*. doi:10.2312/pgv.20221067
- [20] Emil Fransson, Jonatan Hermansson, and Yan Hu. 2024. A Comparison of Performance on WebGPU and WebGL in the Godot Game Engine. In *Proceedings of IEEE Gaming, Entertainment, and Media Conference*. doi:10.1109/GEM61861.2024.10585437
- [21] Henri Gilbert and Helena Handschuh. 2003. Security Analysis of SHA-256 and Sisters. In *Proceedings of International Workshop on Selected Areas in Cryptography*. doi:10.1007/978-3-540-24654-1_13
- [22] Harrison Green and Jacob D. Durrant. 2021. DeepFrag: An Open-Source Browser App for Deep-Learning Lead Optimization. *Journal of Chemical Information and Modeling* 61, 6 (2021), 2523–2529. doi:10.1021/acs.jcim.1c00103
- [23] Robert Haase, Loic A. Royer, Peter Steinbach, Deborah Schmidt, Alexandr Dibrov, Uwe Schmidt, Martin Weigert, Nicola Maghelli, Pavel Tomancak, Florian Jug, and Eugene W. Myers. 2019. CLIJ: GPU-Accelerated Image Processing for Everyone. *Nature Methods* 17 (2019), 5–6. doi:10.1038/s41592-019-0650-1
- [24] Yudong Han, Weichen Bi, Ruibo An, Deyu Tian, Qi Yang, and Yun Ma. 2025. GL2GPU: Accelerating WebGL Applications via Dynamic API Translation to WebGPU. In *Proceedings of ACM on Web Conference*. doi:10.1145/3696410.3714785
- [25] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. 2008. CHStone: A Benchmark Program Suite for Practical C-Based High-Level Synthesis. In *Proceedings of IEEE International Symposium on Circuits and Systems*. doi:10.1109/ISCAS.2008.4541637
- [26] Masatoshi Hidaka, Yuichiro Kikura, Yoshitaka Ushiku, and Tatsuya Harada. 2017. WebDNN: Fastest DNN Execution Framework on Web Browser. In *Proceedings of ACM International Conference on Multimedia*. doi:10.1145/3123266.3129394
- [27] Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. 2019. Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code. In *Proceedings of USENIX Annual Technical Conference*. <https://dl.acm.org/doi/10.5555/3358807.3358817> [accessed on 09/30/2025].
- [28] Daniel Lustig and Margaret Martonosi. 2013. Reducing GPU Offload Latency via Fine-Grained CPU-GPU Synchronization. In *Proceedings of IEEE Symposium on High-Performance Computer Architecture*. doi:10.1109/HPCA.2013.6522332
- [29] Yun Ma, Dongwei Xiang, Shuyu Zheng, Deyu Tian, and Xuanzhe Liu. 2019. Moving Deep Learning into Web Browser: How Far Can We Go?. In *Proceedings of ACM on Web Conference*. doi:10.1145/3308558.3313639
- [30] Edward Meeds, Remco Hendriks, Said Al Faraby, Magiel Bruntink, and Max Welling. 2015. MLitB: Machine Learning in the Browser. *PeerJ Computer Science* 1, 11 (2015). doi:10.48550/arXiv.1412.2432
- [31] Juan-J. Merelo, Antonio Mora-Garcia, JIJ Laredo, Juan Lupión, and Fernando Tricas. 2007. Browser-Based Distributed Evolutionary Computation: Performance and Scaling Behavior. In *Proceedings of Annual Conference Companion on Genetic and Evolutionary Computation*. doi:10.1145/1274000.1274083
- [32] John Michalakos and Manish Vachharajani. 2008. GPU Acceleration of Numerical Weather Prediction. In *Proceedings of IEEE International Symposium on Parallel and Distributed Processing*. doi:10.1109/IPDPS.2008.4536351
- [33] Javad Nejati and Aruna Balasubramaniam. 2016. An In-Depth Study of Mobile Browser Performance. In *Proceedings of ACM International Conference on World Wide Web*. doi:10.1145/2872427.2883014
- [34] Saga Palmér. 2024. *Performance Comparison of WebGPU and WebGL for 2D Particle Systems on the Web: An Analysis of GPU Time in Web-Based Graphics APIs*. M.Sc. Thesis. KTH School of Electrical Engineering and Computer Science, Stockholm, Sweden. <https://www.diva-portal.org/smash/get/diva2:1945245/FULLTEXT02.pdf> [accessed on 09/30/2025].
- [35] Fazli Sapuan, Matthew Saw, and Eugene Cheah. 2018. General-Purpose Computation on GPUs in the Browser Using GPU.js. *IEEE Computing in Science and Engineering* 20, 1 (2018), 33–42. doi:10.1109/MCSE.2018.108163116
- [36] S. Sarathi and P. S. Varshiga. 2024. Unleashing the Power of Graphics: A Comprehensive Exploration of WebGPU. *International Research Journal on Advanced Engineering and Management* 2, 05 (2024). doi:10.47392/IRJAEM.2024.0265
- [37] Kristian Sons, Felix Klein, Dmitri Rubinstein, Sergiy Byelozorov, and Philipp Slusallek. 2010. XML3D – Interactive 3D Graphics for the Web. In *Proceedings of International Conference on Web 3D Technology*. doi:10.1145/1836049.1836076
- [38] Antero Taivalsaari, Tommi Mikkonen, Dan Ingalls, and Krzysztof Palacz. 2008. Web Browser as an Application Platform. In *Proceedings of Euromicro Conference on Software Engineering and Advanced Applications*. doi:10.1109/SEAA.2008.17
- [39] Z. Usta. 2024. WebGPU: A New Graphic API for 3D WebGIs Applications. In *Proceedings of International Conference on Geoinformation Advances*. doi:10.5194/isprs-archives-XLVIII-4-W9-2024-377-2024
- [40] Richard Vuduc, Aparna Chandramowlishwaran, Jee Choi, Murat (Efe) Guney, and Aashay Shringarpure. 2010. On the Limits of GPU Acceleration. In *Proceedings of USENIX Conference on Hot Topics in Parallelism*. <https://dl.acm.org/doi/10.5555/1863086.1863099> [accessed on 09/30/2025].
- [41] Qipeng Wang, Shiqi Jiang, Zhenpeng Chen, Xu Cao, Yuanchun Li, Aoyu Li, Yun Ma, Ting Cao, and Xuanzhe Liu. 2025. Anatomizing Deep Learning Inference in Web Browsers. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–43. doi:10.1145/3688843
- [42] Yutian Yan, Tengfei Tu, Lijian Zhao, Yuchen Zhou, and Weihang Wang. 2021. Understanding the Performance of WebAssembly Applications. In *Proceedings of ACM Internet Measurement Conference*. doi:10.1145/3487552.3487827

Appendix

A Ethics

This work does not raise any ethical issues.